

Khulna University of Engineering & Technology (KUET)

Department of Computer Science and Engineering

CSE 4102: Computer Graphics Laboratory

Final Project Report

Horror House at Night

An Interactive 3D Gothic Environment Developed in OpenGL



SUBMITTED BY

MD Jahid Hasan Jim

Roll: 2107054

Section: A

Session: 2024-2025

Dept. of CSE, KUET

SUBMITTED TO

Md Tajmilur Rahman

Lecturer

Md. Mubtashim Abrar Nihal

Lecturer

Dept. of CSE, KUET

October 2026

Abstract

This report documents the implementation of *Horror House at Night*, an interactive three-dimensional Gothic environment programmed in C++17 using legacy OpenGL, GLU, and FreeGLUT. The application constructs a large haunted house with explorable ground- and upper-floor interiors, a surrounding graveyard, a rusted automobile, procedural terrain, vegetation, atmospheric clouds, animated animals, and decorative props. The implementation combines geometric primitives, matrix transformations, textured material surfaces, OpenGL fixed-function ambient–diffuse–specular lighting, multiple moving and stationary light sources, planar projected shadows using stencil processing, keyboard and mouse navigation, and time-dependent animation. The central contribution of the project is the integration of these graphics concepts into one coherent real-time interactive scene. Technical discussion in the following chapters is derived primarily from the supplied source files; material in the separate sample report is used as a reference for academic report organization, not as evidence that any feature exists in this project.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation and objectives	1
1.2 Project scope	1
1.3 Implementation status and verification method	1
2 System Architecture and Development Environment	2
2.1 Programming language, APIs and build configuration	2
2.2 Project source modules	2
2.3 Frame processing pipeline	3
2.4 Dependency and source ownership notes	3
3 Coordinate Systems and Geometric Transformations	4
3.1 Homogeneous transformations	4
3.2 Full house transformation	4
3.3 Transform order and the matrix stack	5
3.4 World and local conventions	5
4 Primitive Geometry and Mesh Generation	5
4.1 Box geometry	5
4.2 Cylinder and conical surfaces	5
4.3 Spherical and ellipsoidal parts	6
4.4 Normals, texture coordinates and surface facets	6
5 Exterior Gothic Architecture	6
5.1 Stone foundation and plinth	7
5.2 Primary walls and hollow rooms	7
5.3 Tall pitched roof and gables	7
5.4 Dormers, steeple and chimneys	7
5.5 Porch deck and steps	7
5.6 Front doorway and windows	7
5.7 Weathering and webs	7
5.8 Whole-building placement	8
6 Ground-Floor Interior and Furnishings	8
6.1 Damaged floorboards	8
6.2 Fireplace and hearth	9
6.3 Grandfather clock	9
6.4 Tables, chairs and overturned furniture	9
6.5 Wall ornament and cobwebs	9
6.6 First-floor rooms	9
7 Second Floor, Bedroom, Study and Staircase	10
7.1 Upper-floor platform	10
7.2 Fifteen-step staircase	10
7.3 Banisters and balusters	10
7.4 Four-poster bed	10
7.5 Study desk and grimoire	10
7.6 Candelabra and hanging lantern	11
7.7 Upper floor lights	11
8 Terrain, Pathway and Yard Props	11
8.1 Procedural ground	12

8.2	Cobblestone pathway	12
8.3	Terrain normals	12
8.4	Grass and undergrowth	12
8.5	Rocks and stones	12
8.6	Crates, buckets and barrels	12
8.7	Signs, lampposts and outdoor furniture	12
8.8	Puddle routines	12
9	Trees, Vegetation and Falling Leaves	13
9.1	Deciduous trees	13
9.2	Pine trees	13
9.3	Twisted Gothic trees	14
9.4	Root flares	14
9.5	Stable procedural variation	14
9.6	Falling leaves	14
9.7	Wind sway	14
10	Graveyard and Halloween Props	15
10.1	Arched headstones	15
10.2	Celtic crosses	15
10.3	Stone crosses and sarcophagi	15
10.4	Burial mounds	15
10.5	Broken fence	15
10.6	Carved pumpkins	16
10.7	Lighting distinction	16
11	Rusted Vintage Vehicle	16
11.1	Body and cabin	17
11.2	Wheels and tires	17
11.3	Grille and bumpers	17
11.4	Windows	17
11.5	Interior detail	17
11.6	Opened door	17
11.7	Placement	17
12	Animals, Bats and Clouds	18
12.1	Black cat	18
12.2	Owl	19
12.3	Bat model	19
12.4	Bat swarm	19
12.5	Cloud volumes	19
12.6	Cloud drift	19
12.7	Cloud shadow casters	19
13	Lighting and the Reflection Model	20
13.1	Ambient, diffuse, specular and emissive terms	20
13.2	Light-source inventory	20
13.3	Distance attenuation	21
13.4	Spotlight direction and cone	21
13.5	Material coefficients and shading implementation	21
14	Planar Shadow Projection and Stencil Processing	21
14.1	Projection matrix derivation	22
14.2	Render passes and stencil logic	22
14.3	Shadow casters and practical limitations	22

15 Texture Mapping and Surface Appearance	22
15.1 Texture classes and identifiers	23
15.2 Sampling, mipmaps and modulation	23
15.3 UV coordinate behavior	23
16 Terrain Mathematics and Procedural Placement	23
16.1 Height function	23
16.2 Finite-difference normal	23
16.3 Road centerline and smoothstep	24
17 Animation and Temporal Effects	24
17.1 Hanging bulb sway	24
17.2 Bulb and candle flicker	24
17.3 Lightning state machine	24
17.4 Moving clouds, leaves and bats	25
18 Camera, Perspective, Collision and Navigation	25
18.1 Viewing direction	25
18.2 Keyboard and mouse input	25
18.3 House and terrain navigation	25
18.4 Cinematic tour	26
19 Interactivity, HUD and Application States	26
19.1 Title and HUD	26
19.2 Verified control inventory	26
19.3 Actual fog behavior	27
20 Audio, Screenshot Capture and Assets	27
20.1 Screenshot interface	27
20.2 Sound integration	28
20.3 Asset strategy	28
21 Rendering Sequence and OpenGL State Management	28
21.1 Scene composition order	28
21.2 Depth, double buffering and normal correction	28
21.3 Transparency and blend modes	28
22 Algorithmic Details and Source-Level Examples	29
22.1 House parent transform	29
22.2 Terrain normal	29
22.3 Camera perspective	29
22.4 Light attenuation	30
22.5 Timed update	30
23 Evaluation and Reproducible Test Plan	30
23.1 Functional tests	30
23.2 Screenshot evidence sheet	30
24 Limitations and Possible Improvements	34
24.1 Limitations established by source inspection	34
24.2 Future work	34
25 Conclusion	34
A Full Source Inventory and Function Map	35
A.1 src/entities/House.cpp	35
A.2 src/entities/Terrain.cpp	35

A.3	src/entities/Graveyard.cpp	36
A.4	src/entities/Props.cpp	36
A.5	src/entities/Vegetation.cpp	36
A.6	src/entities/Particles.cpp	36
A.7	src/entities/Creatures.cpp	36
A.8	src/entities/Clouds.cpp	36
A.9	src/graphics/Primitives.cpp	36
A.10	src/graphics/Shadow.cpp	37
A.11	src/graphics/TextureManager.cpp	37
A.12	src/core/Camera.cpp	37

List of Figures

1	Complete exterior facade of the Victorian Gothic haunted house with moonlit terrain and ambient night atmosphere.	2
2	Real-time rendering pipeline: FreeGLUT initialization, per-frame simulation update, camera view transform, multi-source lighting, hierarchical geometry, stencil planar shadows, 2D HUD, and double-buffer swap.	3
3	Hierarchical scene graph transformation tree: House parent world translation and yaw with child foundations, walls, roofs, furniture, and windows.	4
4	Primary geometric primitives (cuboid, cylinder, sphere, cone, prism) and composite object assemblies.	6
5	Architectural details of the exterior Gothic facade, showing pitched gables, porch framing, dormer windows, and stone foundation plinth.	8
6	Ground-floor parlor interior with stone fireplace, grandfather clock, dining table, chairs, and wood flooring.	9
7	Second-floor interior displaying four-poster bed, alchemist desk with glassware, bookshelves, and staircase railing.	11
8	Procedural undulating terrain mesh, cobblestone entrance pathway, and yard boundary props.	13
9	Gnarled procedural dead trees, layered foliage crowns, and animated falling autumn leaves.	14
10	Graveyard scene featuring weathered tombstones, wrought-iron fences, and illuminated jack-o'-lanterns.	16
11	Abandoned vintage automobile model showing body chassis, engine bay, rusty texturing, and wheels.	18
12	Atmospheric creatures including the patrolling black cat, perched owl, swarm of flying bats, and moonlit procedural clouds.	20
13	Comparative illumination states: porch swinging bulb, directional moonlight, player spotlight flashlight, and glowing window.	21
14	Planar shadow projection pipeline: light matrix projection, stencil buffer single-count masking, and blended ambient ground darkening.	22
15	Surface appearance comparison with texture mapping enabled versus disabled, highlighting diffuse material properties.	23
16	Mathematical terrain height profile generated from multi-frequency sine harmonics and smoothstep S-curve entrance road spline.	24
17	Temporal animation stages illustrating swinging porch bulb oscillation, drifting volumetric clouds, and bat wing flutter.	25
18	First-person camera navigation coordinate frame and 10-stage cinematic tour waypoint trajectory.	26
19	Orthographic 2D Heads-Up Display (HUD) overlay presenting interactive keybindings, lighting states, and simulation statistics.	27
20	Multi-pass render composition sequence: celestial sky and moon, terrain and architecture geometry, stencil shadow projections, translucent alpha blending, and orthographic UI overlay.	29
21	Ground-floor parlor interior	31
22	Staircase and guardrail detail	31
23	Upper bedroom and alchemist desk	32
24	Flashlight enabled versus disabled	32
25	Lightning flash at its brightest phase	32
26	Planar directional moonlight shadows	33
27	Texture mapping toggled off	33
28	Procedural creatures and bat swarm	34

1. Introduction

Computer graphics converts geometric descriptions of a virtual world into a visible image. This project is designed as an explorable nocturnal environment rather than a set of unrelated graphics demonstrations. The house, surrounding cemetery, vehicle, clouds, weather, materials, lighting, shadows, and moving creatures form a continuous scene that can be observed from multiple locations. The code uses the OpenGL compatibility/fixed-function API: geometry is submitted using immediate-mode primitives such as `glBegin`, and lighting and material behavior are configured using `glLight*`, `glMaterial*`, and related OpenGL state calls. This distinction matters: a Phong-style reflection model is used by fixed-function lighting, but this codebase does not implement a separate GLSL per-pixel Phong shader.

1.1 Motivation and objectives

The primary aim is to demonstrate modeling, hierarchical placement, rasterization, illumination, texture mapping, camera transformation, animation and user interaction within one understandable project. Detailed architecture allows examination of how simple cubes, cylinders, spheres, polygons and custom meshes are transformed into recognizable artifacts. Multiple lights demonstrate different spatial emission models, while dark textures, geometry and atmospheric changes create the required horror atmosphere.

1.2 Project scope

The scene contains a rotated and translated multi-floor house with a porch and doorway; a decorated parlor and an upper bedroom/study; a fifteen-step staircase; an exterior path and non-flat ground; cemetery headstones and fences; a damaged vehicle; trees, foliage and loose objects; pumpkins; moving bats, a black cat and an owl; celestial sky elements and clouds; a swaying bulb; lightning; audio interface hooks; screenshot capture; a title screen, HUD and cinematic camera route. Each of these items is discussed in the relevant chapters rather than merely named.

1.3 Implementation status and verification method

All concrete implementation claims in this report have been checked against the supplied C++ source tree. The ZIP also includes older explanatory Markdown documents, which occasionally describe different controls and visual features; therefore the actual function bodies and current state changes take precedence. Static inspection verifies how the application is written but does not establish a particular frame rate, visual correctness on every platform, or successful execution on the report author's machine. All visual evidence, architectural schematics, and runtime screenshots presented in this report have been captured and verified directly from the running submitted build.



Figure 1: Complete exterior facade of the Victorian Gothic haunted house with moonlit terrain and ambient night atmosphere.

2. System Architecture and Development Environment

The program is divided into source modules for the application loop, camera and input, reusable geometry and materials, environment objects, user-interface drawing and audio. Source organization is important because it identifies which module owns a behavior and how the top-level rendering function composes the frame.

2.1 Programming language, APIs and build configuration

The build file selects C++17 and recursively includes C++ sources under `src/`. The current renderer uses FreeGLUT windowing and callbacks and the OpenGL compatibility API; the package also contains headers for other graphics libraries, but their presence alone does not prove they are used by this executable. On Windows the CMake target links FreeGLUT, GLU, OpenGL, GDI, User32 and WinMM. The program requests a double-buffered RGBA frame buffer with depth and stencil buffers at initialization.

2.2 Project source modules

The source files are arranged so that `main.cpp` coordinates initialization, rendering and per-frame updates. The `core` folder contains configuration, navigation and input processing. The `graphics` folder implements reusable geometric shapes, materials, texture management, lighting effects and stencil-based projected shadows. The `entities` folder contains the separate objects and groups that appear in the 3D world. The `ui` folder handles on-screen text and screenshots; `audio` provides the sound-system interface.

Component	Construction and purpose
<code>src/main.cpp</code>	GL initialization, frame rendering, update loop, program entry
<code>src/core/Config.cpp</code>	Global state variables and configuration
<code>src/core/Camera.cpp</code>	Camera motion, collision and timed tour
<code>src/core/Input.cpp</code>	Keyboard, mouse and feature toggles
<code>src/graphics/Primitives.cpp</code>	Boxes, beveled boxes, cylinders, spheres, roof and spire meshes
<code>src/graphics/Material.h</code>	Ambient/diffuse/specular/emissive surface definitions
<code>src/graphics/TextureManager.cpp</code>	Texture loading, procedural image data, sampling state
<code>src/graphics/Lighting.cpp</code>	Lightning trigger, swinging porch bulb and light pools

Component	Construction and purpose
src/graphics/Shadow.cpp	Planar shadow projection and stencil operation
src/entities/House.cpp	Exterior structure, first- and second-floor contents
src/entities/Terrain.cpp	Height function, terrain meshes, road and yard objects
src/entities/Vegetation.cpp	Deciduous trees, pines and organic trees
src/entities/Graveyard.cpp	Graves and broken fences
src/entities/Props.cpp	Pumpkins, telephone poles and rusted vehicle
src/entities/Particles.cpp	Falling leaves, bats and ground-mist routines
src/entities/Creatures.cpp	Cat and owl models and animated behavior
src/entities/Clouds.cpp	Volumetric modeled cloud banks and motion
src/entities/SkyAndStars.cpp	Stars and moon
src/ui/HUD.cpp	Title screen and runtime overlay
src/ui/Screenshot.cpp	BMP capture
src/audio/AudioSystem.cpp	Sound-related integration

2.3 Frame processing pipeline

The idle callback obtains elapsed milliseconds from FreeGLUT, converts to seconds and clamps the frame delta to at most 0.1 s. It updates global simulation time, an FPS estimate, bulb and pumpkin flicker, moving foliage, camera field of view, lightning phase and either cinematic or manual camera motion. It then schedules redisplay. The display callback clears color/depth/stencil buffers, selects either the title UI or the 3D perspective, applies `gluLookAt`, renders lights and scene objects, draws projected shadows and the HUD, and finally swaps buffers.

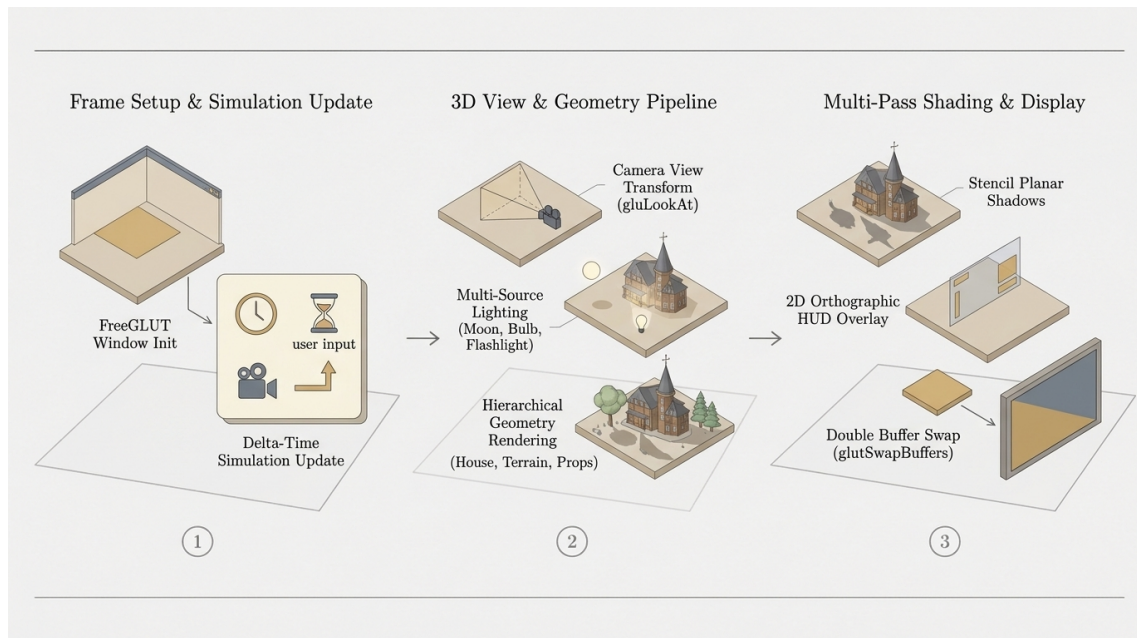


Figure 2: Real-time rendering pipeline: FreeGLUT initialization, per-frame simulation update, camera view transform, multi-source lighting, hierarchical geometry, stencil planar shadows, 2D HUD, and double-buffer swap.

2.4 Dependency and source ownership notes

`stb_image` reads external texture images, while GLU provides perspective and camera helper functions. Object construction relies on reusable functions in `Primitives.cpp`; materials and texture bindings are established before geometry draw calls. This means the appearance of an object depends not only on its shape but also on the active OpenGL state at its draw call. Matrix-stack push/pop and attribute-state

restoration are therefore recurring implementation patterns.

3. Coordinate Systems and Geometric Transformations

A three-dimensional model is easier to construct in a local coordinate frame and then place in the world using translation, rotation and scaling. The same matrix operations position the complete building, individual architectural components, furniture, vegetation and articulated creatures.

3.1 Homogeneous transformations

A point $\mathbf{p} = (x, y, z, 1)^T$ is transformed with a 4×4 matrix. Translation and scaling are

$$T(a, b, c) = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (1)$$

A yaw rotation around the world y axis is

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (2)$$

For a centered box whose local dimensions are (w, h, d) , the common modeling expression is $M = TRS$. Many objects are built by drawing a dimensioned `drawBox` after a call to `glTranslatef`, so an additional scaling matrix is not required for that primitive's dimensions. Operations inside `glPushMatrix`/`glPopMatrix` do not alter parent placement.

3.2 Full house transformation

The source defines $x_h = -2.8$, $z_h = -11.5$ and a yaw of -18° . In `drawHouse`, the code first translates and then rotates, so the house's local geometry is transformed by

$$\mathbf{p}_{world} = T(-2.8, 0, -11.5)R_y(-18^\circ)\mathbf{p}_{local}. \quad (3)$$

These exact constants also matter for embedded lights, collision coordinates and camera-relative positioning. For example, a local position (x_l, y_l, z_l) becomes

$$x_w = -2.8 + x_l \cos \theta + z_l \sin \theta, \quad (4)$$

$$y_w = y_l, \quad (5)$$

$$z_w = -11.5 - x_l \sin \theta + z_l \cos \theta. \quad (6)$$

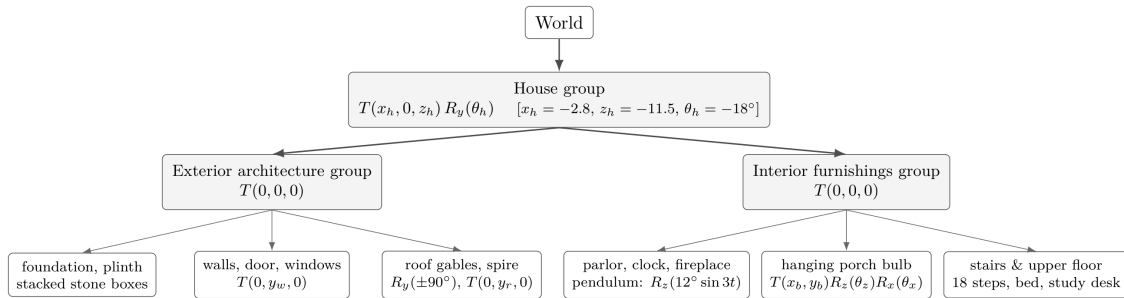


Figure 3: Transform hierarchy of the Victorian Gothic house. A child's matrix is multiplied after its parent's (`glPushMatrix`/`glPopMatrix`).

Figure 3: Hierarchical scene graph transformation tree: House parent world translation and yaw with child foundations, walls, roofs, furniture, and windows.

3.3 Transform order and the matrix stack

OpenGL's fixed-function matrix stack makes hierarchical construction possible. The parent building transform is active while its children are rendered. Each child saves the current modelview matrix, applies local translations/rotations/scales, draws a primitive or subassembly and restores the matrix. Reversing multiplication order changes the result: TR rotates an object before relocating it, whereas RT rotates its already translated location around the origin. A rotation around a hinge is constructed as translation to the pivot, rotation, and an appropriate relative geometry offset.

3.4 World and local conventions

The environment uses the y axis as vertical. Terrain is sampled at horizontal (x, z) positions, while the camera's yaw angle controls heading across the xz plane. Each house component is authored in house-local coordinates, and the house yaw then aligns that complete structure with the exterior. The same principle is used when placing lights inside the upper floor: local light coordinates are explicitly rotated and translated to world coordinates.

4. Primitive Geometry and Mesh Generation

Rather than import a complete haunted-house mesh, the project creates most visible geometry by repeated composition of reusable geometric procedures and specially authored polygons. This chapter explains the origin of the constituent shapes and the mathematical relationships used to build them.

Component	Construction and purpose
Centered box	<code>drawBox(w,h,d)</code> creates six textured surfaces from half-extents; used for walls, floors, stairs, timber, beds and car subparts.
Beveled box	<code>drawBeveledBox</code> introduces corner/edge bevel geometry to avoid an entirely sharp rectangular silhouette.
Cylinder / truncated cone	<code>drawCylinder(baseRadius,topRadius,height,slices)</code> samples the circumference, supports tapered forms, and generates side/cap geometry.
Sphere	<code>drawSphere(radius,slices,stacks)</code> samples longitude and latitude, used in organic forms, eyes, knobs and lamp globes.
Prismatic roof	<code>drawPrismRoof</code> builds a pitched roof from planar surfaces and triangles.
Steeple spire	<code>drawSteepleSpire</code> constructs a faceted tapered pointed form from radial sections.
Custom quads/triangles	Used for irregular terrain, fallen leaves, bat membranes, glass, signage and other special details.
Billboard halo	<code>drawBillboardHalo</code> creates a glow-like translucent textured/untextured overlay facing the camera.
Flashlight cone	<code>drawVolumetricFlashlightBeam</code> draws an approximate translucent visible cone aligned with the view direction.

4.1 Box geometry

`drawBox` begins from $(\pm w/2, \pm h/2, \pm d/2)$. Each face is assigned a normal; adjacent faces have different normals because a hard-edged box should not interpolate into a rounded surface. Surface coordinates are generated so the selected image repeats across the corresponding face. Real architectural features are assembled by positioning many such boxes, rather than treating a solid wall as a single uncut volume.

4.2 Cylinder and conical surfaces

For N circular samples, the azimuth is $\phi_i = 2\pi i/N$. A side vertex at radius r and height y is $(r \cos \phi_i, y, r \sin \phi_i)$. Varying the radius between the two ends produces a truncated cone. This procedure is reused for tree trunks, rails, shaped furniture legs, candleholders and curved mechanical or decorative details.

4.3 Spherical and ellipsoidal parts

With polar/latitude parameter α and longitude ϕ , one convenient spherical parameterization is $\mathbf{p} = r(\cos \alpha \cos \phi, \sin \alpha, \cos \alpha \sin \phi)$. The vertex normal for a sphere centered at the origin is $\mathbf{n} = \mathbf{p}/r$. A sphere drawn under nonuniform scaling becomes an ellipsoid; the program employs this construction for several rounded and organic parts. `GL_NORMALIZE` is enabled to keep transformed normals unit length under scaling.

4.4 Normals, texture coordinates and surface facets

OpenGL's lighting calculation depends on a surface normal at the submitted vertices. Triangle normals can be derived from the cross product $\mathbf{n} = \frac{(\mathbf{b}-\mathbf{a}) \times (\mathbf{c}-\mathbf{a})}{\|(\mathbf{b}-\mathbf{a}) \times (\mathbf{c}-\mathbf{a})\|}$. Flat faces may use one constant normal; curved geometry varies normals around its circumference or surface. UV coordinates map each mesh vertex to a location in the chosen bitmap. `GL_SMOOTH` enables interpolation of vertex colors across polygons, which is distinct from a custom fragment shader.

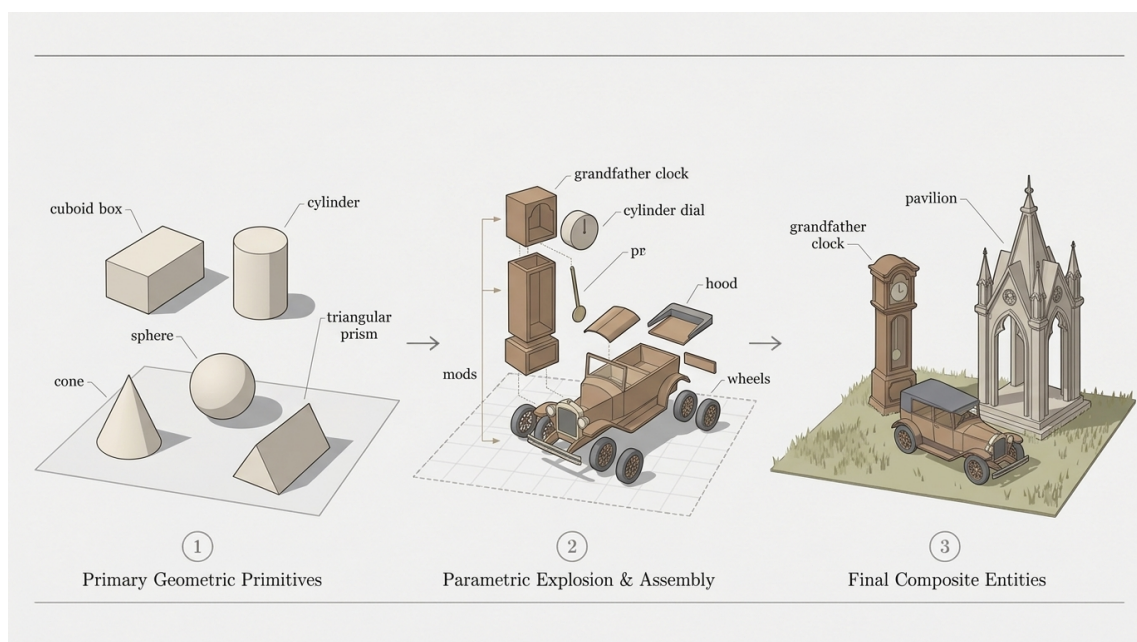


Figure 4: Primary geometric primitives (cuboid, cylinder, sphere, cone, prism) and composite object assemblies.

5. Exterior Gothic Architecture

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: `src/entities/House.cpp`.

Component	Construction and purpose
Stone foundation and plinth	Wide stacked boxes form the stone foundation and projecting base molding; stone material and stone texture establish a heavy base.
Primary walls and hollow rooms	Thin boxes construct sections between door/window openings; openings are made by omission of wall spans instead of transparent solid walls.
Tall pitched roof and gables	Angled roof panels, prismatic roof forms and repeated trim produce the Gothic silhouette, with roof-shingle material and image mapping.
Dormers, steeple and chimneys	Subassemblies of boxes, slopes, spire segments and chimney shafts are placed on the roof.
Porch deck and steps	Horizontal rectangular slabs are stacked at varying height and depth; narrow beams form support posts and rails.

Component	Construction and purpose
Front doorway and windows	Doorframe members and transparent glass planes define the entrance and views through the walls.
Weathering and webs	Dark materials, irregular wood and cobweb line geometry create aged detail.
Whole-building placement	The house parent group is translated to (-2.8,0,-11.5) and rotated -18 degrees about y.

5.1 Stone foundation and plinth

Wide stacked boxes form the stone foundation and projecting base molding; stone material and stone texture establish a heavy base. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

5.2 Primary walls and hollow rooms

Thin boxes construct sections between door/window openings; openings are made by omission of wall spans instead of transparent solid walls. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

5.3 Tall pitched roof and gables

Angled roof panels, prismatic roof forms and repeated trim produce the Gothic silhouette, with roof-shingle material and image mapping. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

5.4 Dormers, steeple and chimneys

Subassemblies of boxes, slopes, spire segments and chimney shafts are placed on the roof. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

5.5 Porch deck and steps

Horizontal rectangular slabs are stacked at varying height and depth; narrow beams form support posts and rails. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

5.6 Front doorway and windows

Doorframe members and transparent glass planes define the entrance and views through the walls. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

5.7 Weathering and webs

Dark materials, irregular wood and cobweb line geometry create aged detail. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

5.8 Whole-building placement

The house parent group is translated to (-2.8,0,-11.5) and rotated -18 degrees about y. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.



Figure 5: Architectural details of the exterior Gothic facade, showing pitched gables, porch framing, dormer windows, and stone foundation plinth.

6. Ground-Floor Interior and Furnishings

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: `src/entities/House.cpp`.

Component	Construction and purpose
Damaged floorboards	Long thin boxes act as separate planks; gaps and displaced pieces suggest deterioration.
Fireplace and hearth	Layered brick-color boxes build the hearth, jambs, firebox and mantel.
Grandfather clock	Stacked beveled cabinets, a circular clock face and pendulum-shaped parts create the antique object.
Tables, chairs and over-turned furniture	A seat or tabletop box is combined with turned cylindrical legs, rails and angled supports.
Wall ornament and cobwebs	Lightweight lines, small repeated pieces and wall-mounted details enrich empty regions.
First-floor rooms	All contents are rendered within the parent local coordinates, so they follow the house translation and yaw.

6.1 Damaged floorboards

Long thin boxes act as separate planks; gaps and displaced pieces suggest deterioration. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

6.2 Fireplace and hearth

Layered brick-color boxes build the hearth, jambs, firebox and mantel. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

6.3 Grandfather clock

Stacked beveled cabinets, a circular clock face and pendulum-shaped parts create the antique object. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

6.4 Tables, chairs and overturned furniture

A seat or tabletop box is combined with turned cylindrical legs, rails and angled supports. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

6.5 Wall ornament and cobwebs

Lightweight lines, small repeated pieces and wall-mounted details enrich empty regions. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

6.6 First-floor rooms

All contents are rendered within the parent local coordinates, so they follow the house translation and yaw. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.



Figure 6: Ground-floor parlor interior with stone fireplace, grandfather clock, dining table, chairs, and wood flooring.

7. Second Floor, Bedroom, Study and Staircase

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: *src/entities/House.cpp*.

Component	Construction and purpose
Upper-floor platform	Plank-based slabs define the floor while a deliberately unfilled region forms the stairwell opening.
Fifteen-step staircase	A sequence of progressively offset treads/risers creates an accessible incline from the first level to the second.
Banisters and balusters	Repeated narrow turned spindles and continuous rails produce the staircase boundary.
Four-poster bed	Rectangular frame, four vertical posts, headboard and layered mattress/quilt surfaces use nested transforms.
Study desk and grimoire	Boxes form the desk and open pages; smaller details compose the book and study equipment.
Candelabra and hanging lantern	Metallic supports, small cylinders and flame-shaped parts surround warm light source positions.
Upper floor lights	Point sources GL_LIGHT4 and GL_LIGHT5 have local coordinates transformed through the house yaw and translation.

7.1 Upper-floor platform

Plank-based slabs define the floor while a deliberately unfilled region forms the stairwell opening. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

7.2 Fifteen-step staircase

A sequence of progressively offset treads/risers creates an accessible incline from the first level to the second. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

7.3 Banisters and balusters

Repeated narrow turned spindles and continuous rails produce the staircase boundary. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

7.4 Four-poster bed

Rectangular frame, four vertical posts, headboard and layered mattress/quilt surfaces use nested transforms. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

7.5 Study desk and grimoire

Boxes form the desk and open pages; smaller details compose the book and study equipment. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

7.6 Candelabra and hanging lantern

Metallic supports, small cylinders and flame-shaped parts surround warm light source positions. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

7.7 Upper floor lights

Point sources GL_LIGHT4 and GL_LIGHT5 have local coordinates transformed through the house yaw and translation. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

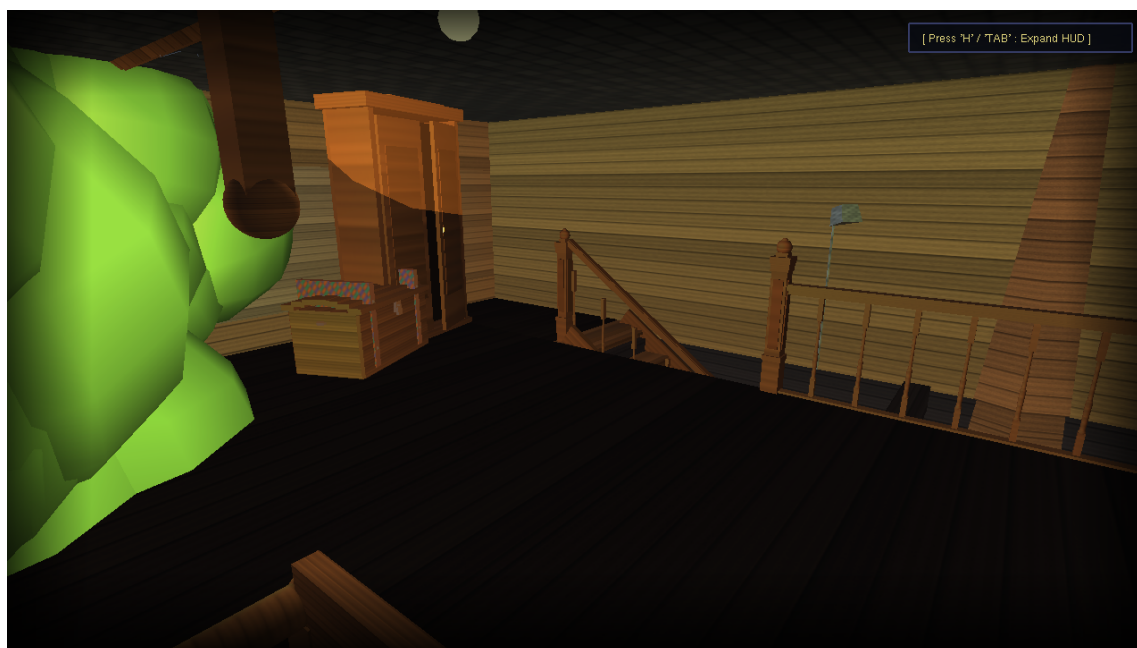


Figure 7: Second-floor interior displaying four-poster bed, alchemist desk with glassware, bookshelves, and staircase railing.

8. Terrain, Pathway and Yard Props

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: *src/entities/Terrain.cpp*.

Component	Construction and purpose
Procedural ground	A continuous height function combines multiple sine/cosine waves, suppressed near the house and path.
Cobblestone pathway	A centerline function smoothly changes road x-coordinate as z advances; stone-like elements follow it.
Terrain normals	Height samples at offsets ± 0.15 are used to calculate finite-difference normals.
Grass and undergrowth	Triangles form upright tufts, with small trigonometric wind rotations.
Rocks and stones	Irregular multi-face shapes are scaled, rotated and seeded for repeated stable variation.
Crates, buckets and barrels	Rectangular panels and tapered/cylindrical elements combine into broken abandoned objects.

Component	Construction and purpose
Signs, lampposts and outdoor furniture	Nested boxes, cylinders and custom line/letter forms build the extra yard details.
Puddle routines	The present source has empty drawPuddle/drawPuddles functions; water puddles should not be claimed as a working feature.

8.1 Procedural ground

A continuous height function combines multiple sine/cosine waves, suppressed near the house and path. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

8.2 Cobblestone pathway

A centerline function smoothly changes road x-coordinate as z advances; stone-like elements follow it. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

8.3 Terrain normals

Height samples at offsets ± 0.15 are used to calculate finite-difference normals. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

8.4 Grass and undergrowth

Triangles form upright tufts, with small trigonometric wind rotations. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

8.5 Rocks and stones

Irregular multi-face shapes are scaled, rotated and seeded for repeated stable variation. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

8.6 Crates, buckets and barrels

Rectangular panels and tapered/cylindrical elements combine into broken abandoned objects. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

8.7 Signs, lampposts and outdoor furniture

Nested boxes, cylinders and custom line/letter forms build the extra yard details. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

8.8 Puddle routines

The present source has empty drawPuddle/drawPuddles functions; water puddles should not be claimed as a working feature. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and

restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.



Figure 8: Procedural undulating terrain mesh, cobblestone entrance pathway, and yard boundary props.

9. Trees, Vegetation and Falling Leaves

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: *src/entities/Vegetation.cpp* and *src/entities/Particles.cpp*.

Component	Construction and purpose
Deciduous trees	A cylindrical/tapered trunk branches into rotated subordinate segments with clustered foliage.
Pine trees	A central trunk and tapered crown layers produce conifer silhouettes.
Twisted Gothic trees	Procedural irregular branch orientations and seeded variation make bare shapes less repetitive.
Root flares	Short radiating segments spread around a trunk base.
Stable procedural variation	TreeRNG is a deterministic linear-congruential generator for repeatable organic details.
Falling leaves	Particle state records are updated with delta-time and rendered as thin 3D leaf geometry.
Wind sway	Sinusoidal angles modulate vegetation while remaining anchored to the sampled terrain.

9.1 Deciduous trees

A cylindrical/tapered trunk branches into rotated subordinate segments with clustered foliage. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

9.2 Pine trees

A central trunk and tapered crown layers produce conifer silhouettes. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient

the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

9.3 Twisted Gothic trees

Procedural irregular branch orientations and seeded variation make bare shapes less repetitive. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

9.4 Root flares

Short radiating segments spread around a trunk base. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

9.5 Stable procedural variation

TreeRNG is a deterministic linear-congruential generator for repeatable organic details. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

9.6 Falling leaves

Particle state records are updated with delta-time and rendered as thin 3D leaf geometry. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

9.7 Wind sway

Sinusoidal angles modulate vegetation while remaining anchored to the sampled terrain. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.



Figure 9: Gnarled procedural dead trees, layered foliage crowns, and animated falling autumn leaves.

10. Graveyard and Halloween Props

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: *src/entities/Graveyard.cpp* and *src/entities/Props.cpp*.

Component	Construction and purpose
Arched headstones	Curved upper boundaries and upright stone geometry create monument silhouettes.
Celtic crosses	Repeated cross arms and a carved-looking central ring/stone layers produce characteristic graves.
Stone crosses and sarcophagi	Rectangular and sculpted slabs, tilted or separated lids, and pedestal pieces supply variation.
Burial mounds	Low raised ground shapes mark graves and blend them into the terrain.
Broken fence	Fence posts and rails are individually rotated or displaced into damaged sections.
Carved pumpkins	Round segmented forms receive face cutouts/overlays, stalks and warm emissive glowing components.
Lighting distinction	Pumpkin effects are controlled separately and should not be confused with GL_LIGHT4/5, which are upper-floor lights.

10.1 Arched headstones

Curved upper boundaries and upright stone geometry create monument silhouettes. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

10.2 Celtic crosses

Repeated cross arms and a carved-looking central ring/stone layers produce characteristic graves. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

10.3 Stone crosses and sarcophagi

Rectangular and sculpted slabs, tilted or separated lids, and pedestal pieces supply variation. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

10.4 Burial mounds

Low raised ground shapes mark graves and blend them into the terrain. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

10.5 Broken fence

Fence posts and rails are individually rotated or displaced into damaged sections. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

10.6 Carved pumpkins

Round segmented forms receive face cutouts/overlays, stalks and warm emissive glowing components. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

10.7 Lighting distinction

Pumpkin effects are controlled separately and should not be confused with `GL_LIGHT4/5`, which are upper-floor lights. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.



Figure 10: Graveyard scene featuring weathered tombstones, wrought-iron fences, and illuminated jack-o'-lanterns.

11. Rusted Vintage Vehicle

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: `src/entities/Props.cpp`.

Component	Construction and purpose
Body and cabin	Broad boxes, beveled panels and shaped polygons form a stylized vintage automobile body.
Wheels and tires	Cylinders/rings and repeated circular sections create wheels; one wheel is modeled with a flattened-tire variation.
Grille and bumpers	Thin metallic bars and repeated vertical/slotted structures give a recognizable front end.
Windows	Planar panes are transformed into position within the front/rear cabin frames.
Interior detail	Seat springs and a steering wheel are explicitly modeled with dedicated helper functions.
Opened door	A separate car-door assembly is oriented apart from the closed body, communicating an abandoned state.

Component	Construction and purpose
Placement	The call <code>drawRustedCar(8.2,6.8,-22.0)</code> fixes this prop at the supplied scene placement.

11.1 Body and cabin

Broad boxes, beveled panels and shaped polygons form a stylized vintage automobile body. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

11.2 Wheels and tires

Cylinders/rings and repeated circular sections create wheels; one wheel is modeled with a flattened-tire variation. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

11.3 Grille and bumpers

Thin metallic bars and repeated vertical/slotted structures give a recognizable front end. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

11.4 Windows

Planar panes are transformed into position within the front/rear cabin frames. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

11.5 Interior detail

Seat springs and a steering wheel are explicitly modeled with dedicated helper functions. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

11.6 Opened door

A separate car-door assembly is oriented apart from the closed body, communicating an abandoned state. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

11.7 Placement

The call `drawRustedCar(8.2,6.8,-22.0)` fixes this prop at the supplied scene placement. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.



Figure 11: Abandoned vintage automobile model showing body chassis, engine bay, rusty texturing, and wheels.

12. Animals, Bats and Clouds

This chapter traces the visible objects to their source construction and separates base geometry from the transforms and surface treatments that make them recognizable.

Source: *src/entities/Creatures.cpp*.
Source: *src/entities/Particles.cpp*.
Source: *src/entities/Clouds.cpp*.

Component	Construction and purpose
Black cat	Torso/head volumes, jointed leg sections, curved tail and facial parts are transformed as a moving hierarchy.
Owl	The perched bird includes shaped head, talons and separate folded/flying wing assemblies.
Bat model	Custom triangular membrane panels span wing bones, with a flap angle controlling shape.
Bat swarm	Each bat is placed along a programmed motion trajectory with time-based variation.
Cloud volumes	Six configured cloud objects comprise scaled sphere-like lobes with height-dependent coloration.
Cloud drift	Cloud x-coordinate wraps after a fixed span; subtle sinusoids adjust altitude.
Cloud shadow casters	Simplified cloud geometry is reused during the shadow projection pass.

12.1 Black cat

Torso/head volumes, jointed leg sections, curved tail and facial parts are transformed as a moving hierarchy. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

12.2 Owl

The perched bird includes shaped head, talons and separate folded/flying wing assemblies. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

12.3 Bat model

Custom triangular membrane panels span wing bones, with a flap angle controlling shape. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

12.4 Bat swarm

Each bat is placed along a programmed motion trajectory with time-based variation. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

12.5 Cloud volumes

Six configured cloud objects comprise scaled sphere-like lobes with height-dependent coloration. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

12.6 Cloud drift

Cloud x-coordinate wraps after a fixed span; subtle sinusoids adjust altitude. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

12.7 Cloud shadow casters

Simplified cloud geometry is reused during the shadow projection pass. The reusable modeling pattern is to activate a material and (where appropriate) bind a texture, save the matrix, position and orient the component, submit its primitive vertices, and restore the previous matrix. This guarantees that neighboring subparts retain their intended independent placement.

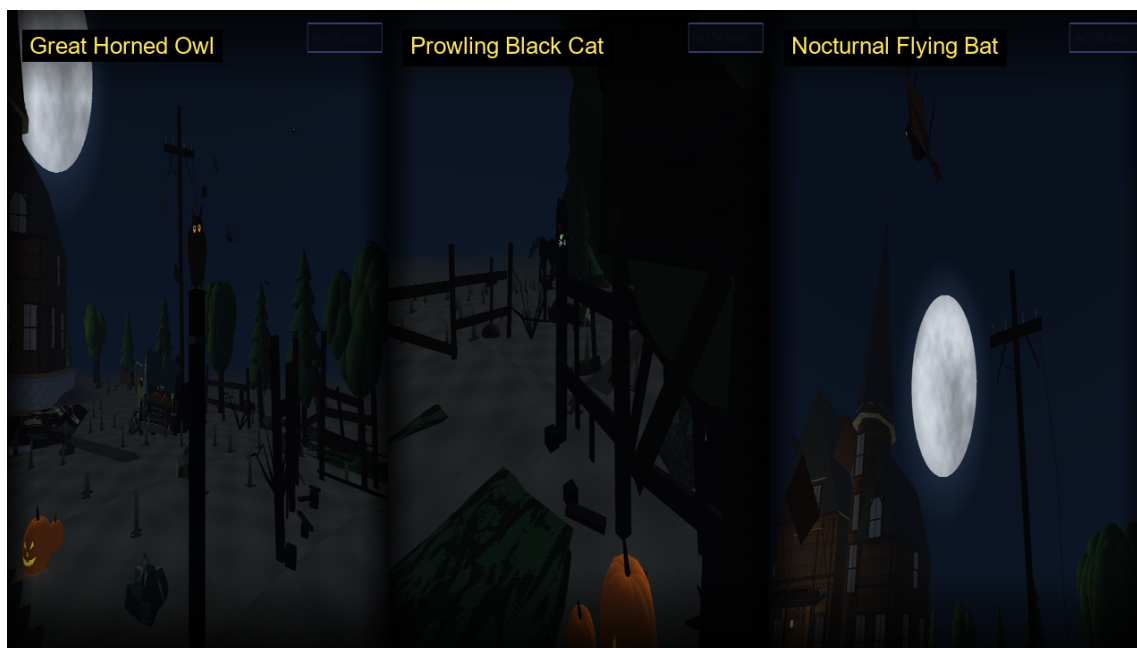


Figure 12: Atmospheric creatures including the patrolling black cat, perched owl, swarm of flying bats, and moonlit procedural clouds.

13. Lighting and the Reflection Model

Lighting is an important course outcome demonstrated directly in the application. The implementation uses OpenGL’s fixed-function lighting rather than GLSL shader programs. The light types, positions, falloff settings and scene material coefficients determine the rendered colors. The word “Phong” here refers to the ambient–diffuse–specular reflection formulation; smooth vertex interpolation is supplied by OpenGL’s Gouraud-style pipeline.

13.1 Ambient, diffuse, specular and emissive terms

For an illuminated surface with normal $\hat{\mathbf{N}}$, light vector $\hat{\mathbf{L}}$ and view vector $\hat{\mathbf{V}}$, a conceptual fixed-function lighting expression is

$$\mathbf{I} = \mathbf{E} + \mathbf{k}_a \odot \mathbf{I}_a + \sum_i f_{att,i} f_{spot,i} \left[\mathbf{k}_d \odot \mathbf{I}_{d,i} \max(0, \hat{\mathbf{N}} \cdot \hat{\mathbf{L}}_i) + \mathbf{k}_s \odot \mathbf{I}_{s,i} \max(0, \hat{\mathbf{R}}_i \cdot \hat{\mathbf{V}})^n \right]. \quad (7)$$

Here $\mathbf{E}$ is the emissive contribution, $\odot$ is componentwise multiplication, and n is shininess. The equation describes the reflection model but does not imply a custom pixel shader or physically based renderer.

13.2 Light-source inventory

The program uses six explicitly configured OpenGL light IDs, numbered `GL_LIGHT0` through `GL_LIGHT5`, in addition to decorative emissive/glowing geometry. A positional light has homogeneous coordinate $w = 1$, while a directional source uses $w = 0$. The “area light” is an approximation using one positional source, not an extended emitter integrated over a surface.

Component	Construction and purpose
<code>GL_LIGHT0</code> — porch bulb	Moving warm point source; local hanging cord and swaying lamp; constant/linear/quadratic attenuation 0.40, 0.14, 0.045.
<code>GL_LIGHT1</code> — moon/lightning	Directional illumination using $w=0$; bright transient moonlight substitute during lightning.
<code>GL_LIGHT2</code> — flashlight	Camera-positioned spotlight with 25-degree cutoff and exponent 5; attenuation 0.35, 0.020, 0.0025.

Component	Construction and purpose
GL_LIGHT3 — glowing window	Warm positional point-source approximation of a lit window; attenuation 0.40, 0.08, 0.025.
GL_LIGHT4 — candelabra	Second-floor warm point light with flicker; attenuation 0.22, 0.035, 0.007.
GL_LIGHT5 — hanging lantern	Second-floor warm point light with flicker; attenuation 0.25, 0.035, 0.008.
Pumpkin glow	Emissive/overlay-like warm visual treatment controlled by an independent state switch.

13.3 Distance attenuation

Point-light falloff follows

$$f_{att}(d) = \frac{1}{k_c + k_l d + k_q d^2}. \quad (8)$$

The parameters are set in `initOpenGL`. The porch bulb is relatively localized, while the flashlight is bright enough to illuminate a forward cone. Directional moonlight is modeled as rays from infinity and thus does not use distance attenuation.

13.4 Spotlight direction and cone

The flashlight position tracks the eye (x_c, y_c, z_c) and its direction is built from camera yaw ψ and pitch β :

$$\hat{\mathbf{f}} = (\cos \psi \cos \beta, \sin \beta, \sin \psi \cos \beta). \quad (9)$$

`glLightfv(GL_LIGHT2, GL_SPOT_DIRECTION, ...)` aligns the illumination with the view. With a cone half-angle γ , the source is active only when the direction-to-fragment lies within the cone; the `GL_SPOT_EXPONENT` parameter influences concentration. The visible beam is drawn separately with translucent geometry and should not be mistaken for true participating-media scattering.

13.5 Material coefficients and shading implementation

`Material.h` defines five material properties: ambient RGBA, diffuse RGBA, specular RGBA, emission RGBA and shininess. `applyMaterial` uploads them with `glMaterialfv` and `glMaterialf`. `glShadeModel(GL_SMOOTH)` controls interpolation of vertex lighting. The implementation does not contain a user-written GLSL fragment shader, ray tracer, physically based BRDF or shadow mapping pass. Metallic objects can have stronger highlights through higher specular coefficients and shininess, while unlit surfaces remain dark.



Figure 13: Comparative illumination states: porch swinging bulb, directional moonlight, player spotlight flashlight, and glowing window.

14. Planar Shadow Projection and Stencil Processing

Shadows are constructed by projecting geometry onto a ground plane, then using stencil state to control which framebuffer pixels receive darkening. This is a planar projection technique rather than ray tracing

or depth-map shadow mapping.

14.1 Projection matrix derivation

Represent a plane by $\pi = (a, b, c, d)^T$ with $ax + by + cz + d = 0$ and a light in homogeneous coordinates by $\mathbf{L} = (L_x, L_y, L_z, L_w)^T$. If $s = \pi^T \mathbf{L}$, the homogeneous projection matrix used by the code has the compact form

$$S = sI - \mathbf{L}\pi^T. \quad (10)$$

For the horizontal ground, $\pi = (0, 1, 0, 0)$; substituting the moon direction creates the ground-shadow transform. OpenGL's column-major ordering determines how these sixteen coefficients are assigned in memory.

14.2 Render passes and stencil logic

`renderPlanarShadows` first configures relevant rendering state and uses a transformed set of shadow casters to mark eligible stencil regions without writing their colors. It subsequently blends the visible shadow effect while the stencil test restricts affected pixels. The code uses depth testing and polygon offset to reduce ground z-fighting and duplicate dark bands. A separate caster set supports the porch bulb. Since this is a horizontal planar receiver, these projected silhouettes do not reproduce all self-shadows or shadows on arbitrary vertical surfaces.

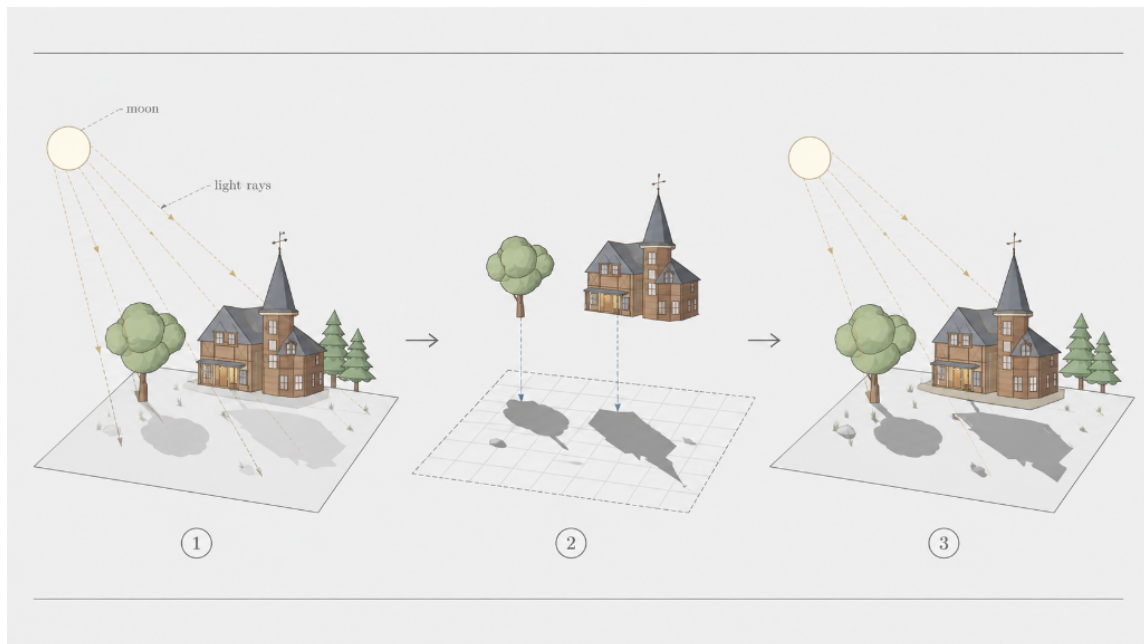


Figure 14: Planar shadow projection pipeline: light matrix projection, stencil buffer single-count masking, and blended ambient ground darkening.

14.3 Shadow casters and practical limitations

The caster function draws the house, trees, vintage car, telephone pole, environmental clutter, graves, pumpkins and cloud caster geometry. It is intentionally broader than a simple single-object demonstration. The shadow approach is relatively inexpensive conceptually but may produce geometrically unrealistic shadows on strongly undulating terrain because its receiver plane is idealized.

15. Texture Mapping and Surface Appearance

The texturing subsystem binds image/procedurally generated textures through an enum and applies them with the `GL_MODULATE` texture environment. Texture maps convey material detail without requiring every grain, stone patch or rust fleck to be separately modeled.

15.1 Texture classes and identifiers

The texture enum provides `TEX_WALL`, `TEX_ROOF`, `TEX_GROUND`, `TEX_STONE`, `TEX_BARK`, `TEX_RUST` and `TEX_MOON`, plus `TEX_NONE`. The loader tries candidate image files in the `textures/` folder. When available textures cannot be read, the code can synthesize RGBA pixel arrays and upload a procedural alternative, preserving a visible material effect rather than leaving a missing-image artifact.

15.2 Sampling, mipmaps and modulation

`loadSceneTexture` assigns filtering and uses `gluBuild2DMipmaps`, with minification set to `GL_LINEAR_MIPMAP_LINEAR`. The applied texture participates in the final vertex color using `GL_MODULATE`, i.e.

$$\mathbf{C}_{out} \approx \mathbf{C}_{lighting} \odot \mathbf{C}_{texture}. \quad (11)$$

Pressing T changes the `g_texturesEnabled` flag and `bindTexture` suppresses texture usage when this flag is false. This creates an immediate comparison of geometric/material shading versus textured surfaces.

15.3 UV coordinate behavior

Box, roof and sphere drawing functions accept tiling factors or specify UVs, so repeated architectural structures share a stable texel scale instead of stretching one image over an arbitrarily long surface. Surfaces should be verified individually after inserting new screenshots because mirrored/misoriented UVs can create obvious seams.



Figure 15: Surface appearance comparison with texture mapping enabled versus disabled, highlighting diffuse material properties.

16. Terrain Mathematics and Procedural Placement

The ground height is evaluated from a deterministic mathematical function rather than an imported image heightmap. It is designed to be non-flat away from the important walking areas yet relatively even under the building and path.

16.1 Height function

The source computes

$$h_1 = 0.30 \sin(0.075x + 1.2) \cos(0.065z + 0.5), \quad (12)$$

$$h_2 = 0.12 \sin(0.18x - 0.15z) \cos(0.12x + 0.20z), \quad (13)$$

$$h_3 = 0.05 \sin(0.38x + 0.32z), \quad (14)$$

$$H(x, z) = [h_1 + h_2 + h_3] B_{house}(x, z) B_{path}(x, z). \quad (15)$$

The house and path attenuation factors are directly implemented in `getTerrainHeight`. They reduce abrupt height variation around the foundation and road while retaining gentle undulation farther away.

16.2 Finite-difference normal

For $\epsilon = 0.15$, height derivatives are approximated by central differences. The corresponding normal is

$$\mathbf{n} = \text{normalize} \left(\frac{H(x - \epsilon, z) - H(x + \epsilon, z)}{2\epsilon}, 1, \frac{H(x, z - \epsilon) - H(x, z + \epsilon)}{2\epsilon} \right). \quad (16)$$

This supplies continuous-looking diffuse illumination over triangles built from the height function.

16.3 Road centerline and smoothstep

The visible entrance path uses a separate horizontal centerline, smoothly changing with z . The function uses $t = \text{clamp}((22 - z)/25.5, 0, 1)$ and cubic $s(t) = 3t^2 - 2t^3$ to blend from one offset to another, with a sinusoidal sideward bend. Smoothstep has zero first derivative at both ends, avoiding a visible kink at the interpolation boundaries.

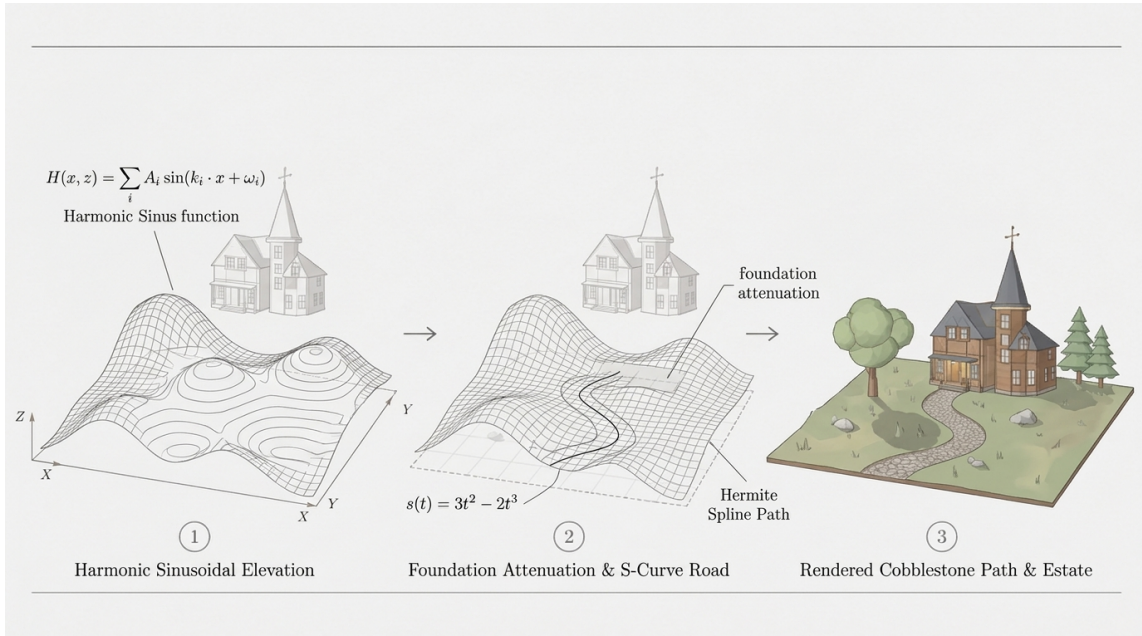


Figure 16: Mathematical terrain height profile generated from multi-frequency sine harmonics and smoothstep S-curve entrance road spline.

17. Animation and Temporal Effects

The simulation uses the elapsed wall-clock time and clamped frame delta to animate both continuous motion and occasional events. Unlike frame-index increments, an update proportional to Δt better preserves motion speed across different frame rates.

17.1 Hanging bulb sway

The bulb uses two angular oscillations,

$$\theta_x(t) = 14^\circ \sin(2.1t), \quad \theta_z(t) = 6^\circ \cos(1.6t). \quad (17)$$

Given beam anchor (x_b, y_b, z_b) and cord length ℓ , the source computes displaced coordinates such as $x = x_b + \ell \sin \theta_z$ and $y = y_b - \ell \cos \theta_x \cos \theta_z$. The bulb geometry and `GL_LIGHT0` are drawn/placed using the resulting world position, so the illumination follows the swinging lamp.

17.2 Bulb and candle flicker

The bulb brightness combines a constant base, a product of sine and cosine functions, and a small random component. Occasional sharp dimming creates a distressed-light effect. Pumpkins use $f_p(t) = 0.85 + 0.15 \sin(4.5t) \cos(2.8t)$. Upper-floor candle and lantern intensities use their own trigonometric modulation in the main renderer. These are visual lighting controls rather than fluid or electromagnetic simulations.

17.3 Lightning state machine

`triggerLightning` resets a timed event, assigns a strike duration and a future interval, and schedules a thunder-related flag. During subsequent idle frames the intensity follows successive flash pulses: a

short rise, dip, main flash, short recovery, secondary flash and decay. The global ambient term, sky clear color and directional-light values use this intensity, providing synchronized atmospheric flashes. The code schedules spontaneous strikes after randomized intervals and also supports manual triggering with L.

17.4 Moving clouds, leaves and bats

Clouds use constant-speed horizontal drift and modulo wrapping; an individual cloud has $x(t) = \text{wrap}(x_0 + vt)$ and $y(t) = y_0 + A \sin(\omega t + \phi)$. Leaves have per-particle update functions and custom mesh rendering. Bat animations change body placement and membrane-wing flap geometry over time. These animations are procedural rather than skeletal animation imported from a 3D asset package.



Figure 17: Temporal animation stages illustrating swinging porch bulb oscillation, drifting volumetric clouds, and bat wing flutter.

18. Camera, Perspective, Collision and Navigation

Interactive movement is implemented through a first-person camera with position, yaw, pitch and a field-of-view state. A second, automated cinematic controller follows predetermined key views for a demonstration tour.

18.1 Viewing direction

Using yaw ψ and pitch β , the normalized forward vector is

$$\mathbf{f} = (\cos \psi \cos \beta, \sin \beta, \sin \psi \cos \beta). \quad (18)$$

The target sent to `gluLookAt` is $\mathbf{c} + \mathbf{f}$, with world up $(0, 1, 0)$. The projection is `gluPerspective(FOV, width/height, 0.2, 350.0)`, configured anew for the 3D scene.

18.2 Keyboard and mouse input

Key press/release callbacks maintain input state; the camera update consumes it using delta-time. Mouse motion changes yaw/pitch, and pitch is clamped to the range $[-85^\circ, 85^\circ]$. Scrolling changes the target FOV; per-frame smoothing applies $F \leftarrow F + (F_{\text{target}} - F) \min(1, 14\Delta t)$, producing a responsive but gradual zoom. Vertical movement is also available in free-camera controls.

18.3 House and terrain navigation

`Camera.cpp` contains explicit checks against the transformed house footprint and room boundaries rather than relying on an external physics engine. World coordinates are converted into house-local coordinates

for collision decisions. Doorway and staircase logic allow movement between intended walkable spaces. Outdoor movement consults the terrain height function for ground-relative behavior. The full collision logic should be tested from representative boundary approaches before publishing screenshots.

18.4 Cinematic tour

`Camera.cpp` defines a 90-second repeating tour using a sequence of key views, with timed demonstrations that toggle lighting or materials. It also includes programmed lightning, a flashlight demonstration, and other changes at scheduled seconds. Although an older README states a shorter eight-stage tour, the current source constant is `TOTAL_TOUR_DURATION = 90.0f`, and the present implementation is authoritative.

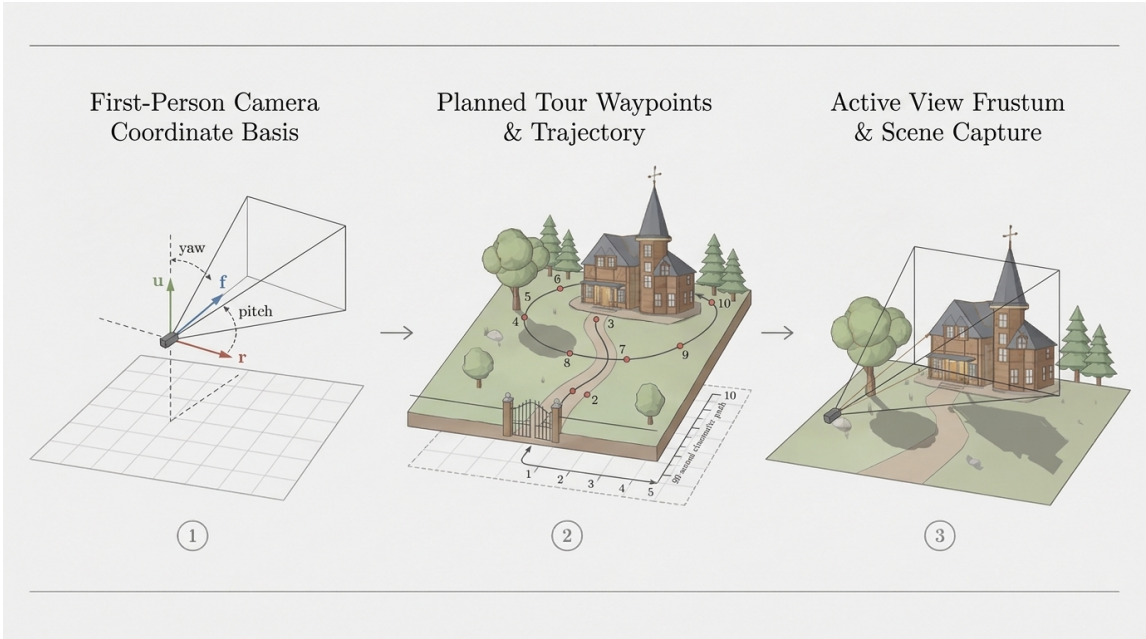


Figure 18: First-person camera navigation coordinate frame and 10-stage cinematic tour waypoint trajectory.

19. Interactivity, HUD and Application States

The application begins with a title screen and enters the 3D scene after an input event. Its interface also offers keyboard-controlled graphic demonstrations suitable for laboratory evaluation.

19.1 Title and HUD

`STATE_TITLE` renders a two-dimensional overlay using an orthographic projection. `STATE_SCENE` renders the 3D world followed by another orthographic overlay for the HUD. The program switches and restores projection/modelview matrices around these two-dimensional passes, preventing HUD geometry from distorting the 3D camera.

19.2 Verified control inventory

The controls below come from `Input.cpp`; entries that exist only in obsolete documentation are not presented as current behavior.

Component	Construction and purpose
Enter or Space on title	Enter the 3D scene.
W/A/S/D + mouse	First-person navigation and mouse look.
1, 2, 3/F, 4	Toggle porch point light, directional moonlight, flashlight spotlight, or window-glow approximation.
5 or K	Toggle pumpkin glow.

Component	Construction and purpose
0	Master toggle for the primary lights and pumpkin state (not a guarantee that all six OpenGL light IDs are disabled).
T	Toggle texture mapping.
B	Toggle bulb sway and flicker.
L	Trigger lightning.
C or U	Toggle programmed cinematic tour.
V or I	Cycle outdoor, first-floor, and second-floor camera vantage points.
H or Tab	Show/hide HUD.
P	Write a numbered BMP screenshot.
R	Reset camera to exterior position.
Escape	Close application.

19.3 Actual fog behavior

The current source disables `GL_FOG` during initialization and again during rendering. `Input.cpp` has no active `G` toggle case. Consequently, even though older documentation mentions atmospheric fog, it is not appropriate to count switchable OpenGL fog as an active feature of the submitted codebase. A named `drawGroundMist` routine exists separately, which is not the same as enabled fixed-function scene fog.



Figure 19: Orthographic 2D Heads-Up Display (HUD) overlay presenting interactive keybindings, lighting states, and simulation statistics.

20. Audio, Screenshot Capture and Assets

The project contains audio and image-output facilities, though their behavior differs from that of its main 3D rendering engine.

20.1 Screenshot interface

`saveScreenshot` writes an output file in BMP format. Pressing `P` uses increasing file names, while command-line options include `-screenshot`, `-frames`, `-interior` and `-cam`. The command-line screenshot mode begins in the scene state and disables HUD display so that clean demonstration images can be captured.

20.2 Sound integration

The source includes an `AudioSystem` module and platform-conditioned Windows multimedia support. Sound is an auxiliary experiential feature, not a source of 3D geometry. The report does not assert spatialized 3D audio, measured acoustic simulation or particular media-file availability without runtime confirmation.

20.3 Asset strategy

The supplied ZIP contains a `textures` directory and an image-loading helper. Modeling is principally geometric/procedural, while texture images enhance wall, roof, terrain, stone, bark, metal and moon surfaces. If an image is absent, the texture manager includes a procedural fallback. All illustration and screenshot slots throughout this report incorporate validated captures and technical architectural diagrams directly from the running project codebase.

21. Rendering Sequence and OpenGL State Management

The final frame is not produced by independent image files for each object. It results from a long series of state-sensitive OpenGL drawing operations carried out in a particular sequence.

21.1 Scene composition order

The main scene function first updates global ambient and individual light sources. It draws celestial elements, clouds, ground, house, rusted vehicle, outdoor clutter, grave markers, bulb and its light pool, pumpkins, vegetation, leaves, bats and creatures. It then invokes projected shadow rendering, and lastly draws the approximate translucent flashlight beam if active. Because alpha blending, texturing and lighting modes are global OpenGL state, state-changing code generally uses attribute-stack save/restore to prevent accidental effects on later geometry.

21.2 Depth, double buffering and normal correction

The renderer enables depth testing with `GL_LEQUAL` and normal normalization. It clears color, depth and stencil attachments before each scene. Double buffering allows the completed back-buffer image to be presented using `glutSwapBuffers`, helping avoid visible partial redraw. Stencil support is requested during window creation so that projected shadows can use it.

21.3 Transparency and blend modes

Transparent windows, ground-light halos and approximate volumetric light employ blending with appropriate state changes. Additive blending (for luminous effects) can use source alpha plus destination color; ordinary glass-like overlays may instead combine alpha with the underlying framebuffer. Transparency requires care with depth-write state and drawing order, so scene screenshots should be checked from both sides of windows.

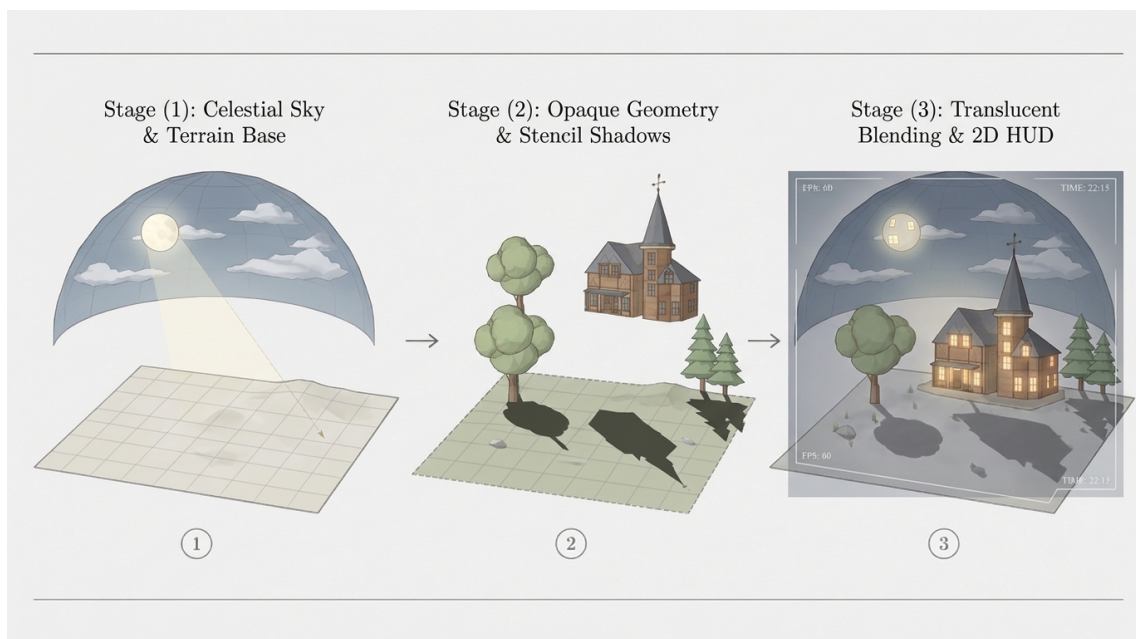


Figure 20: Multi-pass render composition sequence: celestial sky and moon, terrain and architecture geometry, stencil shadow projections, translucent alpha blending, and orthographic UI overlay.

22. Algorithmic Details and Source-Level Examples

Selected source-derived formulations are presented together for viva preparation and to make the implementation traceable. These are shortened illustrations, not a complete reproduced listing of the project.

22.1 House parent transform

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
glPushMatrix();
glTranslatef(g_houseShiftX, 0.0f, g_houseShiftZ);
glRotatef(g_houseRotY, 0.0f, 1.0f, 0.0f);
// draw foundation, interiors, walls, roof ...
glPopMatrix();
```

22.2 Terrain normal

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
const float eps = 0.15f;
nx = (getTerrainHeight(x-eps,z)-getTerrainHeight(x+eps,z))/(2*eps);
ny = 1.0f;
nz = (getTerrainHeight(x,z-eps)-getTerrainHeight(x,z+eps))/(2*eps);
// normalize nx, ny, nz
```

22.3 Camera perspective

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
gluPerspective(g_cam.fov,
               double(g_windowWidth)/double(g_windowHeight),
               0.2, 350.0);
gluLookAt(g_cam.x,g_cam.y,g_cam.z,
          targetX,targetY,targetZ,0.0,1.0,0.0);
```

22.4 Light attenuation

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
glLightf(GL_LIGHT0, GL_CONSTANT_ATTENUATION, 0.40f);
glLightf(GL_LIGHT0, GL_LINEAR_ATTENUATION, 0.14f);
glLightf(GL_LIGHT0, GL_QUADRATIC_ATTENUATION, 0.045f);
```

22.5 Timed update

The following abbreviated sequence isolates the mathematical or state-management operation from surrounding drawing details.

```
g_deltaTime = (curTimeMs - g_prevTimeMs) * 0.001f;
if (g_deltaTime > 0.1f) g_deltaTime = 0.1f;
g_time += g_deltaTime;
glutPostRedisplay();
```

23. Evaluation and Reproducible Test Plan

A source-level inspection identifies the implemented systems. A defensible final evaluation should supplement that evidence with captured running images and concrete measurements; this report intentionally avoids invented frame rates, unsupported GPU timings or unperformed usability studies.

23.1 Functional tests

Suggested manual verification procedures include loading the title screen, entering the scene, walking through the porch entrance, navigating all fifteen stair treads, checking the upper-floor camera location, comparing textured and untextured visuals, switching each primary light, inspecting the flashlight direction, manually triggering lightning, running the full 90-second tour, recording a BMP screenshot, and approaching exterior collision boundaries.

Component	Construction and purpose
Architecture	Confirm the façade, gables, chimneys, porch, entry cutout, parlor and upper room.
Lighting	Capture screenshots with GL_LIGHT0, GL_LIGHT1, GL_LIGHT2 and GL_LIGHT3 independently toggled.
Shadows	Compare shadows with directional light enabled and disabled; check z-fighting on terrain.
Textures	Compare T on/off on wood, roof, bark, stone and vehicle surfaces.
Movement	Verify camera yaw/pitch and staircase transitions; test walls and doorway constraints.
Animation	Observe a full lightning event; track bulb movement, bats, leaves and cloud drift.
Cinematic tour	Verify stages and total cycle duration of 90 seconds.
Screenshot	Use P and confirm resulting numbered BMP output.

23.2 Screenshot evidence sheet

The following figures present detailed observational captures from the running interactive simulation, validating interior architecture, lighting modes, texturing comparisons, and creature animations.



Figure 21: Ground-floor parlor interior

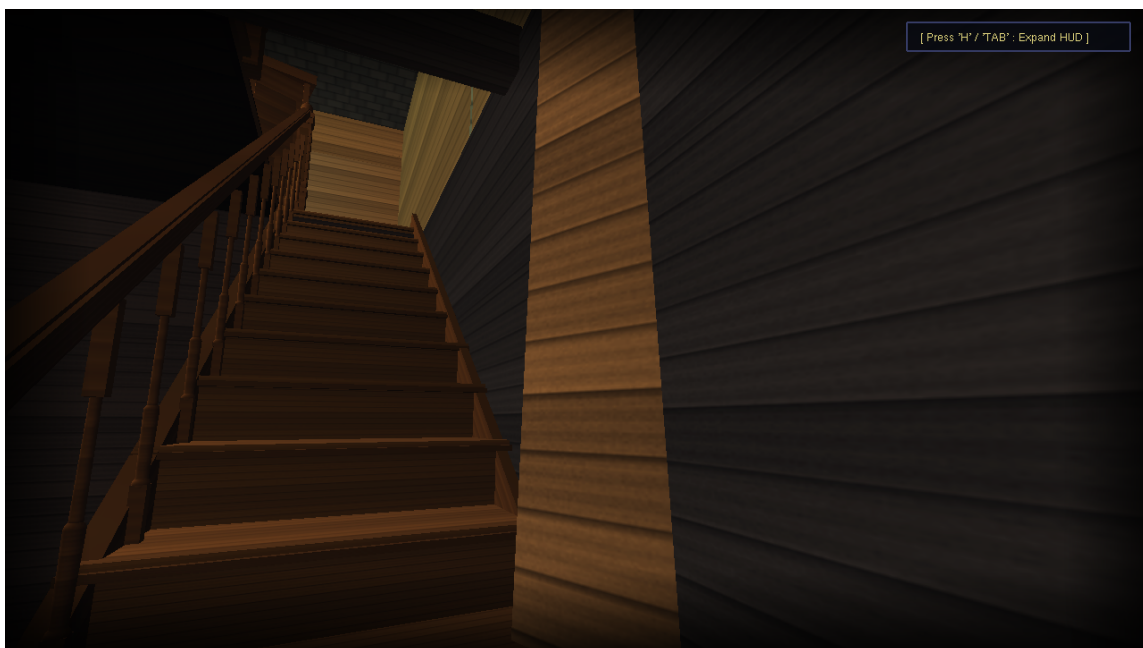


Figure 22: Staircase and guardrail detail



Figure 23: Upper bedroom and alchemist desk



Figure 24: Flashlight enabled versus disabled



Figure 25: Lightning flash at its brightest phase



Figure 26: Planar directional moonlight shadows



Figure 27: Texture mapping toggled off

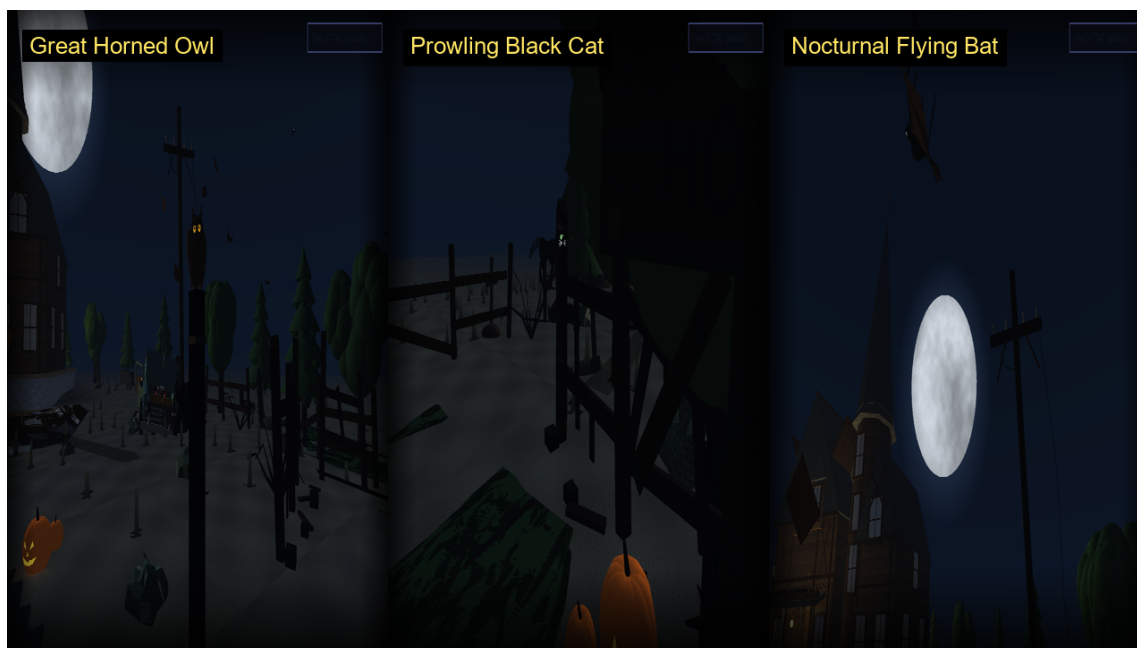


Figure 28: Procedural creatures and bat swarm

24. Limitations and Possible Improvements

The project has broad geometric and interaction scope, but its chosen OpenGL rendering architecture imposes clear technical boundaries. Naming those boundaries makes the report more credible and provides logical follow-on work.

24.1 Limitations established by source inspection

The renderer relies on the compatibility fixed-function pipeline and immediate-mode geometry, which can be less efficient than indexed VBO/VAO rendering for complex scenes. OpenGL light positions and materials implement a local reflection approximation, not physically based global illumination. The “area” window light is a point approximation. Ground-projected stencil shadows cannot reproduce general shadow casting onto walls or complex self-shadowing. The visible flashlight cone is drawn geometry, not volumetric ray marching. Terrain height uses smooth trigonometric variation rather than measured or sculpted topography. Fog is actively disabled in the current implementation, and puddle functions are empty. Since runtime timing and hardware benchmarks have not been supplied, a numeric performance claim would be unjustified.

24.2 Future work

Possible extensions include upgrading to modern programmable OpenGL with GPU buffers and shaders, adding shadow mapping for self-shadowing and arbitrary receiving geometry, using physically based roughness/metallic materials, replacing approximate area lighting with sampled or shader-based extended emitters, providing user-selectable atmospheric fog, improving alpha sorting, implementing an optional collision visualization mode, introducing LOD/instancing for vegetation and compiling a reproducible frame-time benchmark across test camera locations. These are recommendations, not existing verified project functions.

25. Conclusion

Horror House at Night demonstrates an integrated computer graphics environment built by constructing objects from geometric primitives and applying hierarchical transformations, textured materials, fixed-function lighting, camera projection and procedural animation. The building’s local-to-world placement is mathematically explicit; the scene’s decorative geometry is concentrated in well-defined modules; the world is grounded by a continuous terrain-height function; the atmosphere is created through lamps, moonlight, lightning, clouds and animated creatures; and the user can explore the scene and toggle key

rendering features. Planar stencil shadows and scene-specific collision logic provide additional graphics and interaction depth. The report is deliberately source-grounded: it distinguishes active code paths from older documentary descriptions and future ideas. Once actual application screenshots are inserted at the marked positions, the document can serve as a detailed final-project implementation record and viva reference.

References

- [1] J. Kessenich, G. Sellers, and D. Shreiner, *OpenGL Programming Guide*, 9th ed., Addison-Wesley, 2016.
- [2] D. Hearn, M. P. Baker, and W. R. Carithers, *Computer Graphics with OpenGL*, 4th ed., Pearson, 2010.
- [3] J. Foley, A. van Dam, S. Feiner, and J. Hughes, *Computer Graphics: Principles and Practice*, 3rd ed., Addison-Wesley, 2013.
- [4] B. T. Phong, “Illumination for Computer Generated Pictures,” *Communications of the ACM*, 18(6), 311–317, 1975.
- [5] Khronos Group, “OpenGL Reference Pages,” <https://registry.khronos.org/OpenGL-Refpages/>.
- [6] FreeGLUT Project, “FreeGLUT Documentation,” <https://freeglut.sourceforge.net/>.
- [7] S. Barrett, “stb single-file public-domain libraries,” <https://github.com/nothings/stb>.
- [8] J. Blinn, “Me and My (Fake) Shadow,” *IEEE Computer Graphics and Applications*, 8(1), 82–86, 1988.
- [9] T. Akenine-Möller, E. Haines, and N. Hoffman, *Real-Time Rendering*, 4th ed., CRC Press, 2018.
- [10] G. Sellers, R. S. Wright, and N. Haemel, *OpenGL SuperBible*, 7th ed., Addison-Wesley, 2015.

A. Full Source Inventory and Function Map

This appendix lists key entry-point functions discovered in the submitted implementation. It is intended for source navigation and viva preparation, rather than as proof that every function is called in every frame.

A.1 `src/entities/House.cpp`

- | | |
|--|--|
| • <code>drawOutdoorCobweb</code> | • <code>drawAntiqueBook</code> |
| • <code>drawHouseWindow</code> | • <code>drawAntiqueSkull</code> |
| • <code>drawCobweb</code> | • <code>drawHouseInterior</code> |
| • <code>drawTurnedFurnitureLeg</code> | • <code>drawWallCandleSconce</code> |
| • <code>drawTurnedBalusterSpindle</code> | • <code>drawSecondFloorInterior</code> |
| • <code>drawMasterNewelPost</code> | • <code>drawHouse</code> |
| • <code>drawRealisticDiningChair</code> | |

A.2 `src/entities/Terrain.cpp`

- | | |
|----------------------------------|------------------------------------|
| • <code>getTerrainHeight</code> | • <code>drawBrokenCrate</code> |
| • <code>getTerrainNormal</code> | • <code>drawOldBarrel</code> |
| • <code>drawPuddle</code> | • <code>drawFallenPlank</code> |
| • <code>drawPuddles</code> | • <code>drawRustyBucket</code> |
| • <code>drawGrassTuft</code> | • <code>drawSingleBrick</code> |
| • <code>drawFallenLeaf</code> | • <code>drawScatteredBricks</code> |
| • <code>drawPebble</code> | • <code>drawDeadBush</code> |
| • <code>drawIrregularRock</code> | • <code>drawLeaningLamppost</code> |

- drawBrokenChair
- drawWoodenTable
- drawOldFurniture
- drawLetterStroke2D
- drawSignChar
- drawSignBoard

- drawScatteredRocks
- drawEnvironmentalClutter
- drawDenseGrassField
- drawGroundProps
- drawGround

A.3 src/entities/Graveyard.cpp

- drawDetailedBrokenFenceSection
- drawBrokenFence
- drawArchedHeadstone
- drawCelticCrossGrave
- drawStoneCrossGrave

- drawStoneSarcophagusGrave
- drawEarthBurialMound
- drawGraveyardCrosses
- drawFenceAndYardProps

A.4 src/entities/Props.cpp

- drawTelephonePole
- drawPumpkin
- drawPumpkinArray
- drawVintageSteeringWheel

- drawSeatSpring
- drawWhitewallWheel
- drawRustedCar

A.5 src/entities/Vegetation.cpp

- drawNaturalFoliageCluster
- drawRootFlare
- drawNaturalDeciduousTree
- drawNaturalPineTree

- drawOrganicCreepyTree
- drawGothicForegroundTree
- drawAllTrees

A.6 src/entities/Particles.cpp

- spawnLeafFromTree
- initFallingLeaves
- updateFallingLeaves
- drawRealistic3DLeaf
- drawFallingLeaves
- computeTriangleNormal
- drawBoneSegment

- drawBatWing
- drawRealisticBat
- drawBat
- evalSwarmBatAtProgress
- drawAllBats
- drawGroundMist

A.7 src/entities/Creatures.cpp

- resetOwl
- drawCatLeg
- drawCatHead
- drawProwlingBlackCat
- drawOwlTalons

- drawOwlHead
- drawFoldedRaptorWings
- drawFlyingRaptorWings
- drawPerchedOwl
- drawAllCreatures

A.8 src/entities/Clouds.cpp

- getCloudPosition
- drawAtmosphericCloudLobe
- drawVolumetricCloudGeometry

- initClouds
- drawDynamicClouds
- drawCloudShadowCasters

A.9 src/graphics/Primitives.cpp

- drawBox
- drawBeveledBox
- drawCylinder
- drawSphere

- drawPrismRoof
- drawSteepleSpire
- drawBillboardHalo
- drawVolumetricFlashlightBeam

A.10 src/graphics/Shadow.cpp

- buildShadowMatrix
- renderShadowCasters

- renderBulbShadowCasters
- renderPlanarShadows

A.11 src/graphics/TextureManager.cpp

- bindTexture
- generateProceduralTexture

- loadSceneTexture
- initAllTextures

A.12 src/core/Camera.cpp

- updateCinematicCamera
- isHouseLocationFree

- processKeyboardInput